

### **Question 1: Compare and contrast CHAR to VARCHAR datatypes?**

CHAR is a fixed-length data type, meaning it always occupies the full defined storage regardless of the actual content, padding the rest with spaces. VARCHAR, on the other hand, is variable-length and only uses as much space as the actual data needs. CHAR is best for consistent-length data like codes or abbreviations, while VARCHAR suits fields like names or emails where lengths vary.

### **Q2: When to use integer or numeric fields data type? And Why?**

You use INTEGER when dealing with whole numbers like IDs, counts, or ages since it's faster and lighter on storage. NUMERIC or DECIMAL is the go-to when decimals and precision matter, such as prices or grades, because it avoids rounding errors that could mess up calculations, especially in financial data.

### **Q3: What are the differences between UNION and UNION ALL?**

Both combine results from two queries, but UNION automatically removes duplicate rows while UNION ALL keeps everything, including duplicates. Because UNION has to check and filter out duplicates, it runs slower. If you're sure there are no duplicates or you simply need all records, UNION ALL is the better and faster choice.

### **Q4: What are the advantages in using the following DISTINCT, LIMIT, != operator, and <> operator?**

- a.) DISTINCT: DISTINCT is handy when you want only unique values in your results, avoiding cluttered or repeated data.
- b.) LIMIT: LIMIT controls how many rows are returned, which is useful for previewing data or improving query speed.
- c.) != operator: The != operator means "not equal to" and is more commonly seen in MySQL and PostgreSQL, making it a familiar choice for those platforms.
- d.) <> operator: The <> operator works the same way as != but it is the official SQL standard, meaning it works across more database systems. When writing SQL meant to run on multiple platforms, <> is the safer choice.

### **Q5: Discuss how EXPLAIN command optimize the query.**

The EXPLAIN command optimizes queries by acting as a diagnostic tool that reveals the database's execution plan. It allows you to identify performance bottlenecks, specifically full table scans (type: ALL), where the system is forced to read every row because a proper index is missing. By using the output to see which keys are being utilized and how many rows are being processed, you can strategically add indexes to the specific columns identified by the command, transforming slow searches into high-speed lookups and drastically reducing execution time.